Tema 1. Manejo de ficheros en Java

El archivo

- Se considera un archivo (fichero) en informática a un conjunto de datos que se encuentran almacenados en un dispositivo

- Este conjunto de datos viene agrupado por un nombre, una ruta de acceso y una extensión

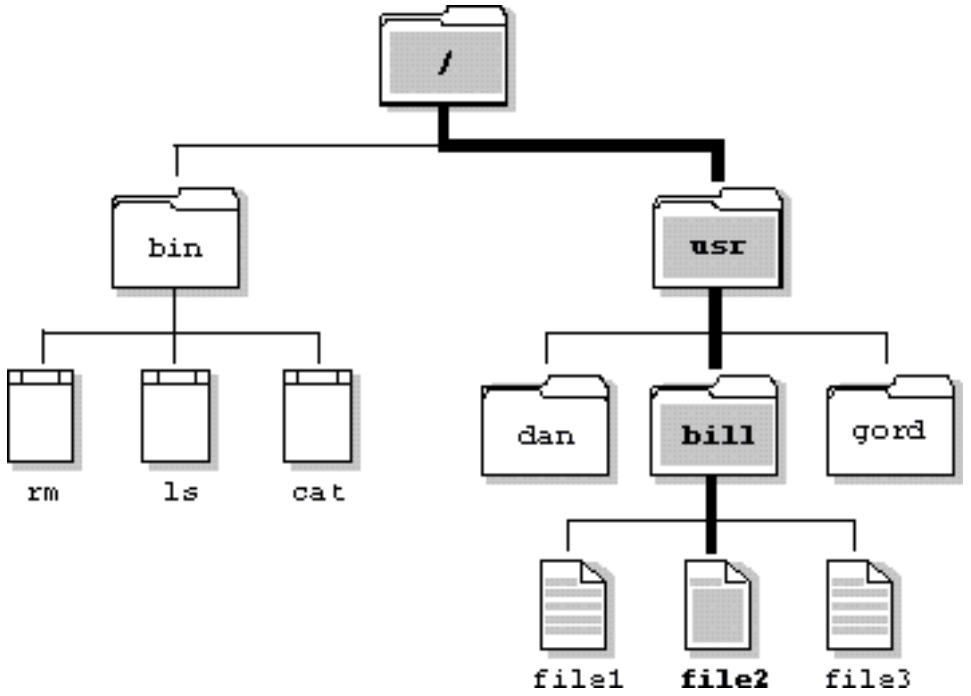
- El conjunto de los 3 elementos debe ser único

- Los ficheros se guardan en dispositivos de almacenamiento fijo como discos duros, pendrives, tarjetas, CD, etc.

Sistema de ficheros

- Los ficheros se suelen organizar de forma jerárquica

- No pueden existir ficheros con el mismo:
  - Nombre
  - Ruta
  - Extensión



Tipos de ficheros



En JAVA, los ficheros pueden clasificarse en dos categorías: ficheros de texto y ficheros binarios. La diferencia está en la forma en que se almacenan y se procesan los datos.

FICHEROS DE TEXTO

- Secuencia de caracteres
- Interpretable por un ser humano
- Generalmente portable
- Escritura/Lectura menos eficientes binarios
- Requiere más tamaño que un fichero binario para representar la misma información
- Ej: Un entero de 9 dígitos en un fichero de texto ocuparía 18 bytes (asumiendo codificación Unicode de 2 bytes/carácter)

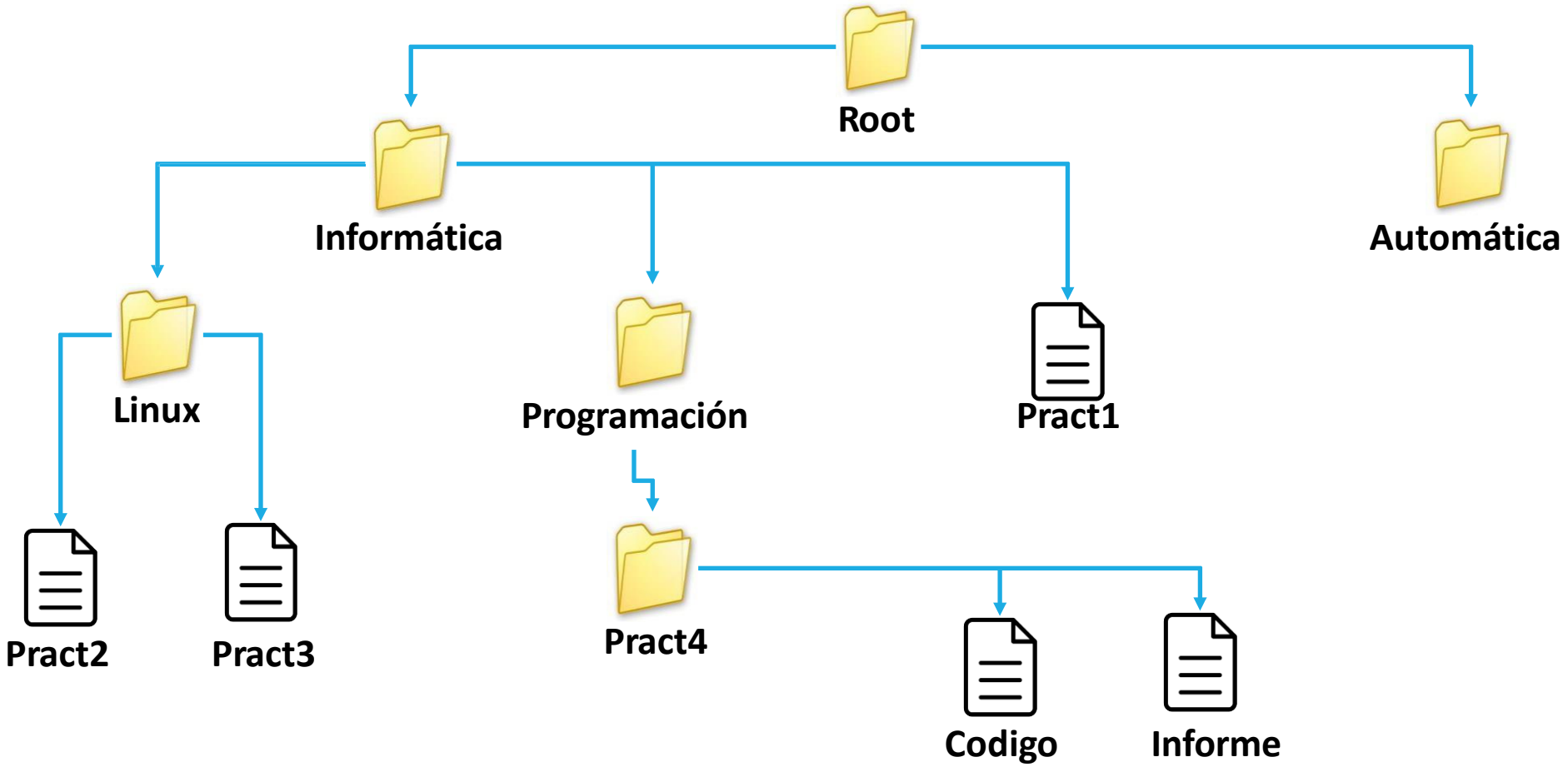
FICHEROS BINARIOS

- Secuencia de bytes (interpretables como tipos primitivos)
- No interpretable por un ser humano
- Generalmente no portable (debe ser leído en el mismo tipo de ordenador y con el mismo lenguaje de programación que fue escrito)
- Escritura/Lectura eficiente
- Almacenamiento eficiente de la información
- Ej: Un entero de 9 dígitos en un fichero binario ocupa 4 bytes

Rutas

- Para acceder a un determinado fichero, se utiliza la **ruta** (path)
- Una ruta indica la dirección del fichero en el sistema de archivos
- En una ruta, cada nivel de la jerarquía se representa delimitado por el símbolo /. En Windows, el símbolo separador es \
- Además de /, existen 2 elementos especiales en la ruta
  - **.** Representa al directorio actual
  - **..** Representa al directorio padre en la jerarquía
- Existen 2 tipos de rutas:
  - **Absoluta**: Ruta al fichero desde el directorio principal (root). **Ej:** */home/Documentos/ejemplo.txt*
  - **Relativa**: Ruta al fichero desde el directorio actual. **Ej:** Estando en home, *./Documentos/ejemplo.txt*

Rutas. Ejemplo de árbol de directorios



Extensión de un fichero

- La extensión sirve para asociar un determinado fichero con un programa/s que lo ejecuten o interpreten
- Indica cómo debe ser interpretado el fichero

doc, docx, odt 

html, xml, jsp 

mp3, avi, wma 

- Se considera extensión lo existente a partir del último punto del nombre del archivo
  - Hola.doc → Extensión = doc
  - Hola.doc.mp3 → Extensión = mp3

Operaciones con ficheros

- **Apertura**: El programa abre el fichero y se prepara para leerlo o escribirlo. Suele “reservar” el fichero para sí
- **Cierre**: Indica que se ha finalizado con las operaciones sobre el fichero. Libera el fichero
- **Lectura**: Lee el fichero o una de sus partes
- **Escritura**: Permite escribir en el fichero, ya sea añadiendo datos o sobrescribiendo los ya existentes
- **Ejecución**: Similar a la lectura, pero utiliza los datos del fichero para ejecutar un software
- **Creación**: Crea un nuevo fichero con un nombre, extensión y ruta determinados
- **Eliminación**: Elimina un fichero determinado

Permisos sobre ficheros

- El usuario que crea el fichero tiene derecho a decidir quien y cómo accede a su fichero

- Existen 3 grupos para los que se les puede definir permisos
  - **u**. Propietario
  - **g**. Grupo
  - **o.** Resto de usuarios

- Los permisos que se pueden dar son los siguientes
  - **r.** Lectura
  - **w**. Escritura
  - **x**. Ejecución

Fichero en Java

- En Java se sigue el concepto de un fichero como un conjunto de bytes
- Por lo tanto, Java puede operar con ficheros de cualquier tipo
- Para encontrarlos, su ruta relativa parte de la ubicación del fichero ejecutable
  - Se permiten también rutas absolutas
- En Java se permiten hacer todas las operaciones con ficheros vistas anteriormente:
  - Apertura y cierre
  - Lectura y escritura
  - Creación y eliminación

Apertura y cierre de un fichero

- Cuando se abre un fichero en Java, se “reserva” el fichero para operar con el
- Se establece un flujo de datos desde el fichero a una variable en Java, que representa al fichero
- A partir de esa variable, se pueden realizar todas las operaciones sobre fichero que se quieran
- Cuando se quiere dejar de usar el fichero, se debe **cerrar** el mismo, cortando el flujo de datos y liberando la variable



La clase File en JAVA

La clase File

- La clase que manipula los ficheros en Java se llama **File**
- Permite hacer un gran número de operaciones sobre un fichero y sus propiedades, pero **no** se permite **leer** ni **escribir**
- También permite obtener datos del fichero, como rutas, nombres, permisos e incluso si existe
- El resto de clases que manipulan ficheros necesitan un objeto de tipo File, por lo que es la base de cualquier operación de manipulación de ficheros.

La clase File: constructor

- El constructor de la clase File tiene el siguiente esquema:

File(String *pathname*): `File variableFichero = new File(“ruta fichero”);`

En Linux: new File("/directorio/fichero.txt");

En Windows: new File("C:\\directorio\\fichero.txt");

File(String *parent*, String *child*):

new File("directorio”, "fichero.txt");

File(File *parent*, String *child*):

new File(new File("directorio"), "fichero.txt")

- La ruta puede ser absoluta o relativa

EJEMPLO 1

La clase File: clases asociadas

Los métodos más utilizados sobre objetos de la clase File son:

- **exists():** Devuelve true si el fichero/directorio existe y false en caso contrario.
- **canWrite():** Devuelve true si se puede escribir en el fichero.
- **canRead():** Devuelve true si se puede leer del fichero.
- **isFile():** Devuelve true si se trata de un fichero y no de un directorio.
- **isDirectory():** Devuelve true si se trata de un directorio y no de un fichero.
- **isAbsolute():** Devuelve true si se trata de un directorio en el path absoluto.
- **lastModified():** Devuelve en milisegundos la fecha de la última modificación. Para convertirlo en fecha normal hay que usar Date.

La clase File: clases asociadas

- **length():** Devuelve el tamaño del fichero en bytes.
- **mkdir():** Crea el directorio indicado.
- **mkdirs():** Crea el directorio indicado incluyendo los subdirectorios.
- **createNewFile():** Crea un nuevo fichero, vacío, asociado a File si no existe un fichero con dicho nombre.
- **renameTo(File *nuevonombre*):** Renombra el fichero con el nuevo nombre.
- **delete():** Borra el fichero o directorio asociado a File.
- **String[ ] list():** Devuelve un array de String con los nombres de los archivos y directorios en el directorio indicado por esta ruta.

Ejemplo de uso de File

```
public static void demoFile(String nombre) throws IOException {

    File fichero = new File( pathname: nombre);
    if (fichero.exists()) {
        System.out.println("Nombre del archivo " + fichero.getName());
        System.out.println("Ruta " + fichero.getPath());
        System.out.println("Ruta absoluta " + fichero.getAbsolutePath());
        System.out.println("Se puede escribir " + fichero.canWrite());
        System.out.println("Se puede leer " + fichero.canRead());
        System.out.println("Tamaño " + fichero.length());
    } else {
        throw new IOException("No se encuentra el fichero " + nombre);
    }
}
```

```
Nombre del archivo: FicheroEjemplo.txt
Ruta: FicheroEjemplo.txt
Ruta absoluta: C:\DirectorioEjemplo\FicheroEjemplo.txt
Se puede escribir: false
Se puede leer: true
Tamaño: 1366
```

EJERCICIO FILE

Borrado de un fichero

- Para borrar un fichero, se utiliza la clase **File**
- Existe la función **delete()** que elimina el fichero

```
public static void main(String args[]) {
   File fichero = new File(“FicheroEjemplo.txt");

   if (fichero.exists()) {
      System.out.println("Nombre del archivo "+ fichero.getName());
      System.out.println(“Ruta "+ fichero.getPath());
      System.out.println(“Ruta absoluta "+ fichero.getAbsolutePath());
      System.out.println("Se puede escribir "+fichero.canWrite());
      System.out.println("Se puede leer "+fichero.canRead());
      System.out.println("Tamaño "+fichero.length());
   }

   fichero.delete();
}
```

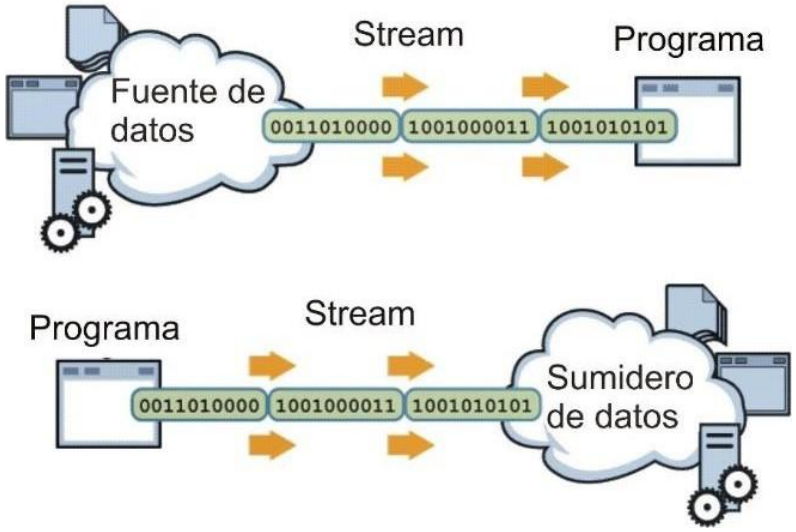
Posibles problemas

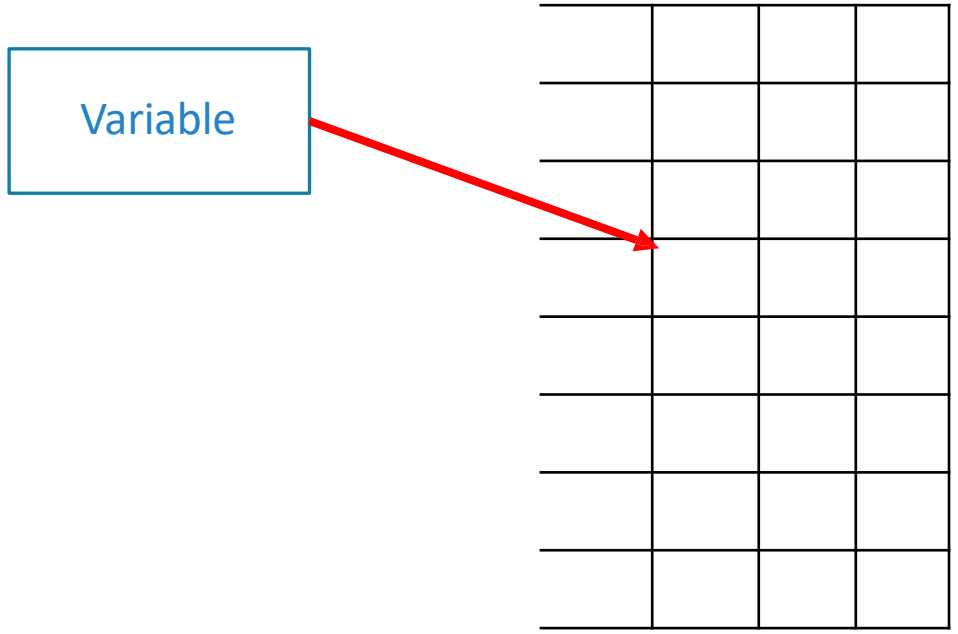
- En las operaciones con ficheros pueden surgir una serie de problemas, que hay que controlar por ejemplo:
  - Intentar leer/borrar/escribir sobre un fichero que no existe
  - Intentar leer/borrar/escribir sobre un fichero que ya está siendo modificado por otro programa
  - Intentar escribir o leer un fichero cuando el puntero haya llegado al final del fichero (EOF)

Flujos y Streams en JAVA

Stream de datos

- Para poder realizar las operaciones de lectura y escritura de ficheros, Java establece lo que se conoce como un Stream de datos
- Tenemos stream carácter (ficheros de texto), y stream bytes (trabaja con todo tipo de ficheros xls, pdf, avi…)
- Crea una vía de comunicación entre programa y fichero que permite “moverse” por las distintas partes del fichero
- Existe un puntero que apunta a las partes del fichero





Flujos y Streams de lectura

Lectura de datos de un fichero

- Para leer datos de un fichero **de texto**, utilizaremos las siguientes clases:
- **Clase File:** Para representar el fichero que se quiere leer

```
File fichero = new File("ruta fichero");
```

- **Clase FileReader:** Establece el stream de datos de lectura del fichero. Tiene una serie de métodos para leer carácter a carácter. Al constructor del FileReader recibe el objeto File

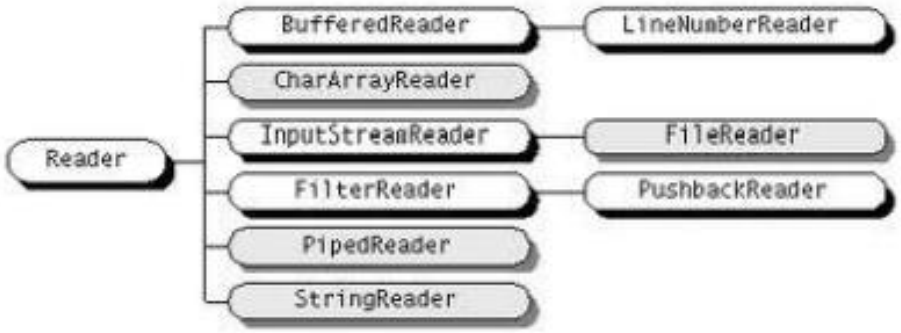
```
FileReader reader = new FileReader(fichero);
```

- **Clase BufferedReader:** Podemos envolver el FileReader y leer línea a línea. Crea un buffer a través del FileReader, que permite leer mas de un carácter. Finaliza la lectura cuando encuentra un salto de línea. El constructor recibe el FileReader como parámetro (o cualquier objeto que que permita leer caracteres: InputStream…)

```
BufferedReader buffer = new BufferedReader (reader);
```

```
datos.txt
   ↓
FileReader
   ↓
BufferedReader
   ↓
programa Java
```

Lectura: la clase abstracta Reader



CLASE	FUNCIÓN
BufferedReader	Permite leer información del buffer intermedio y no directamente al fichero.
CharArrayReader	Lee un flujo de caracteres en un array de caracteres.
InputStreamReader	Convierte el InpuStream en un Reader.
FilterReader	Accede al fichero para lectura.
PipedReader	Permite obtener los datos a una tubería.
StringReader	Es un flujo de caracteres cuya fuente es una cadena de caracteres (String).

Lectura de datos de un fichero

- Función del BufferedReader llamada **readLine()**:
  - Devuelve la siguiente línea de texto si existe
  - Si no existe, devuelve null

```
String linea = buffer.readLine();
```

- Función del BufferedReader llamada **read()**:
  - Lee un caracter
  - Si no existe, devuelve -1

```
String linea = buffer.readLine();
```

- Teniendo en cuenta el funcionamiento de readLine(), se puede leer todo el fichero utilizando un bucle while

```
String linea;
while((linea=buffer.readLine()) != null) {
   System.out.println(linea);
}
```

```
String linea;
linea = buffer.readLine();
while(linea!= null) {
   System.out.println(linea);
   linea = buffer.readLine();
}
```

Lectura con BufferReader

```
class LeerFichero {

    public static void main(String[] arg) {
        try {
            File archivo = new File("TarjetasCredito.txt");
            FileReader reader = new FileReader(archivo);
            BufferedReader buffer = new BufferedReader(reader);

            String linea;
            while ((linea = buffer.readLine()) != null) {
                System.out.println(linea);
            }
            buffer.close();
            //Al cerrar el buffer se cierra automáticamente
            //el FileReader porque esta asociado a el

        } catch (Exception e) {
            e.printStackTrace();
        }

    }
}
```

Lectura con BufferReader OPTIMIZADO

```
class LeerFichero {
    public static void main(String[] args) {
        try (BufferedReader buffer = new BufferedReader(new FileReader("TarjetasCredito.txt"))) {
            String linea;
            while ((linea = buffer.readLine()) != null) {
                System.out.println(linea);
            }
        } catch (IOException e) {
            System.out.println("Error al leer el fichero: " + e.getMessage());
        }
    }
}
```

Ejercicio Biblioteca 1

Problema UTF-8 con caracteres especiales

A la hora de realizar la lectura, FileReader no sabe que codificación tiene que utilizar. En este caso, vemos los caracteres especiales de forma rara. Para solucionarlo:

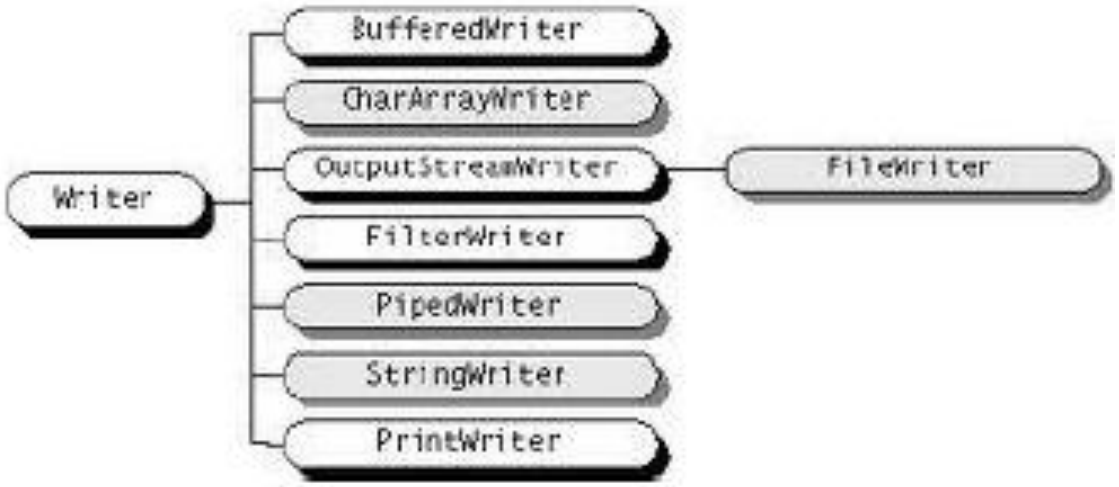
```
FileReader fr = new FileReader("nombres.txt", StandardCharsets.UTF_8);
BufferedReader br = new BufferedReader(fr);
```

Lectura con FileReader

```
public static void demoFileReader(String nombre) {
    try {
        FileReader reader = new FileReader( fileName: nombre);
        int caracter = reader.read();
        char letra;
        while (caracter != -1) {
            letra = (char) caracter;
            System.out.println( x: letra);
            caracter = reader.read();
        }
        reader.close();
    } catch (FileNotFoundException fnfe) { // no se encuentra el fichero
        System.out.println("No se encuentra el fichero " + nombre);
    } catch (IOException ioe) { // hay un error en la lectura del fichero
        System.out.println("Error al leer el fichero " + nombre);
    }
}
```

Flujos y Streams de escritura

Escritura: la clase abstracta Writer



CLASE	FUNCIÓN
BufferedWriter	Permite escribir información del buffer intermedio y no directamente al fichero lo que permite mejorar el rendimiento.
CharArrayWriter	Escribe un flujo de caracteres en un array de caracteres.
OutputStreamWriter	Convierte el OutputStream en un Writer.
FilterWriter	Accede al fichero para escritura.
PipedWriter	Permite obtener los datos a una tubería.
StringWriter	Es un flujo de caracteres cuya salida es un buffer de cadena de caracteres que se usa para generar un String.
PrintWriter	Permite escribir líneas de texto.

Escritura de datos de un fichero

- Para escribir datos en un fichero **de texto**, utilizaremos las siguientes clases:
- Es importante ejecutar close() después de realizar la escritura para asegurarnos de que se graba toda la información en el disco.

- **Clase File:** Para representar el fichero que se quiere leer

```
File fichero = new File("ruta fichero");
```

- **Clase FileWriter:** Permite escribir datos de caracteres en un archivo. Tiene una serie de métodos para escribir en ficheros. Al constructor del FileWriter recibe el objeto File

```
FileWriter writer = new FileWriter(fichero);
```

- **Clase BufferedWriter:** Crea un buffer a través del FileWriter.

```
BufferedWriter pw = new BufferedWriter(writer);
```

Escritura con BufferWriter

Crea un método que reciba el nombre de un archivo, y un entero. El método deberá crear el archivo y escribir 'n' líneas dentro. Cada línea deberá tener escrito "Esta es la línea n", sustituyendo 'n' por el número de la línea.

```
public static void crearLineas(String nombreFichero, int numLineas) {
    try {
        FileWriter fw = new FileWriter(nombreFichero);
        BufferedWriter bw = new BufferedWriter(fw);

        for (int i = 1; i <= numLineas; i++) {
            bw.write("Esta es la línea " + i);
            bw.newLine();
        }

        // Al cerrar el BufferedWriter se cierra también el FileWriter
        bw.close();
        System.out.println(nombreFichero + " creado con " + numLineas + " líneas");

    } catch (IOException e) {
        System.out.println("Error al crear o escribir en el archivo: " + e.getMessage());
    }
}
```

Escritura con BufferWriter OPTIMIZADO

Crea un método que reciba el nombre de un archivo, y un entero. El método deberá crear el archivo y escribir 'n' líneas dentro. Cada línea deberá tener escrito "Esta es la línea n", sustituyendo 'n' por el número de la línea. Utiliza un buffer para realizar la escritura de forma más eficiente.

```
public static void crearLineas(String nombreFichero, int numLineas) {
    try (BufferedWriter bw = new BufferedWriter(new FileWriter(nombreFichero))) {
        for (int i = 1; i <= numLineas; i++) {
            bw.write("Esta es la línea " + i);
            bw.newLine();
        }
        System.out.println(nombreFichero + " creado con " + numLineas + " líneas");
    } catch (IOException e) {
        System.out.println("Error al crear o escribir en el archivo: " + e.getMessage());
    }
}
```

Escritura con FileWriter

- TENER EN CUENTA: El constructor del FileWriter puede recibir un segundo parámetro **boolean** que indica si queremos escribir el fichero desde cero (false) o si queremos añadir texto al existente (true)

```
FileWriter writer = new FileWriter(fichero);
```

```
FileWriter writer = new FileWriter(fichero, false);
```

Desde cero

```
FileWriter writer = new FileWriter(fichero, true);
```

Añadir texto

La clase PrintWriter

- Es una subclase de la clase abstracta Writer, se usa para escribir datos formateados. permite escribir representaciones de varios tipos de datos, incluyendo texto y primitivas como int, long, float, etc. Además, proporciona métodos para imprimir líneas completas (println()), que pueden ser muy útiles. Por ello, PrintWriter es mucho más versátil.

```
PrintWriter pw = new PrintWriter(writer);
```

Método/Constructor	Descripción
PrintWriter(Writer out)	Constructor que crea un objeto PrintWriter utilizando un objeto Writer existente como destino de salida
void print(String s)	Escribe una cadena de caracteres
void println(String s)	Escribe una cadena de caracteres con un salto de línea
void println(tipo_de_dato)	Escribe diferentes tipos de datos(int, double, float, etc.)
void close()	Cierra el flujo de escritura y libera los recursos asociados

Escritura y lectura de datos de un fichero

Las operaciones de lectura y escritura sobre ficheros se pueden combinar de tal forma que haya un flujo de lectura y otro de escritura, uno de lectura y dos de escritura, tres de lectura, etc.

En el ejemplo a continuación hay dos flujos de lectura y uno de escritura. En total tres manejadores de fichero (dos para lectura y uno para escritura). El programa va leyendo, de forma alterna los dos ficheros de origen, una línea de cada fichero (una línea de *origen1.txt* y otra línea de *origen2.txt*) y al mismo tiempo va guardando esas líneas en otro fichero con nombre *mezcla.txt*.

```java
public static void mezclaDosFicheros() {
    try {
        BufferedReader br1 = new BufferedReader(new FileReader( fileName:"origen1.txt"));
        BufferedReader br2 = new BufferedReader(new FileReader( fileName:"origen2.txt"));
        BufferedWriter bw = new BufferedWriter(new FileWriter( fileName:"mezcla.txt"));
        String linea1 = "";
        String linea2 = "";
        while ((linea1 != null) || (linea2 != null)) {
            linea1 = br1.readLine();
            linea2 = br2.readLine();
            if (linea1 != null) {
                bw.write(linea1 + "\n");
            }
            if (linea2 != null) {
                bw.write(linea2 + "\n");
            }
        }
        br1.close();
        br2.close();
        bw.close();
        System.out.println( x:"Archivo mezcla.txt creado satisfactoriamente.");
    } catch (IOException ioe) {
        System.out.println( x:"Se ha producido un error de lectura/escritura");
        System.err.println( x:ioe.getMessage());
    }
```

Procesamiento de archivos de texto

- Cuando se procesa un archivo de texto, los pasos a seguir son los siguientes:
  - 1. Leer una línea del fichero origen mientras quedan líneas por leer.
  - 2. Modificar la línea (utilizando los métodos que ofrece la clase String).
  - 3. Grabar la línea modificada en el fichero destino.
  - 4. Volver al paso 1.
- Algunos métodos de la clase String que puedes utilizar:
  - **`charAt(int n)`** Devuelve el carácter que está en la posición n-ésima de la cadena.
  - **`indexOf(String palabra)`** Devuelve la posición en la que comienza una palabra.
  - **`length()`** Devuelve la longitud de la cadena.
  - **`replace(char c1, char c2)`** Devuelve una cadena en la que se han cambiado todas las ocurrencias del carácter c1 por el carácter c2.
  - **`substring(int inicio,int fin)`** Devuelve una subcadena.
  - **`toLowerCase()`** Convierte todas las letras en minúsculas.
  - **`toUpperCase()`** Convierte todas las letras en mayúsculas.
  - **`split()`** Separa un String en varios Strings y almacenarlos en un array. `String[] fechaSeparada = fecha.split("/");`

Flujos de bytes en Java

Ficheros binarios

- Se utiliza con la serialización.
- Recomendado para envío de ficheros cuando no es necesario su lectura.

- EJEMPLO, lectura de imagen

```
public class LecturaEscrituraBytes {

    public static void main(String[] args) {
        int contadorBytes = 0;

        try {
            FileInputStream archivoLectura = new FileInputStream("./netbeans.png");
            boolean finalAr = false;
            while (!finalAr) {
                int byteEntrada = archivoLectura.read();//lee byte a byte
                if (byteEntrada == -1) finalAr = true;
                System.out.println(byteEntrada);
                contadorBytes++;
            }
            archivoLectura.close();
        } catch (IOException e) {
        }
        System.out.println("La imagen esta formada por este num. de bytes: " + contadorBytes);
    }

}
```

Lectura y escritura de bytes

- **FileInputStream**
- **FileOutputStream**

- Se manejan igual que FileReader y FileWriter estudiados anteriormente

```
FileInputStream fis = new FileInputStream("./netbeans.png");
int byteEntrada = fis.read();          //lee byte a byte
                                       //devuelve -1 cuando ha finalizado
```

```
FileOutputStream fos = new FileOutputStream("./netbeans.png");

fos.write(datosBytes[i]);    //escritura de la posición i de un array de bytes
```

Serialización

Serialización ObjectOutputStream

- La **serialización** de un objeto consiste en convertir en bytes un objeto para su almacenamiento en BD o su transmisión. Después, mediante la **deserialización**, el estado original del objeto se puede reconstruir.

- REQUERIMIENTO → **implemente el interfaz Serializable**. Esta interface no tiene ningún método, así que no se debe proporcionar ningún método adicional.

- Ejemplo, queremos guardar los objetos Empleado de una clase Empresa en un fichero:
  - Construiremos un *ObjectOutputStream* asociado a un *FileOutputStream*.

    ObjectOutputStream salida = new *ObjectOutputStream*(new *FileOutputStream*("personal.ser"));
  - Escribimos el array y cerramos el flujo.

    salida.writeObject(empleados);

    salida.close();

Serialización – Extensiones

- En Java, los ficheros binarios pueden tener cualquier extensión, ya que no afecta el funcionamiento del archivo. Es común por convención:

  `.ser` (de "serialized"): Es la extensión más común cuando se guardan objetos serializados en Java.
  Ejemplo: `clientes.ser, productos.ser`

  `.dat` (de "data"): También es utilizada frecuentemente para archivos binarios que contienen datos generales.
  Ejemplo: `clientes.dat, productos.dat`

```java
public class Empresa {

    private static final String FICHERO = "personal.ser";
    private ArrayList<Empleado> empleados;

    public Empresa() {
        empleados = new ArrayList<>();
    }

public void guardar() {
    try (ObjectOutputStream salida = new ObjectOutputStream(new FileOutputStream(FICHERO))) {
        salida.writeObject(empleados);
        System.out.println("Se han guardado " + empleados.size() + " empleados en " + FICHERO);
    } catch (IOException e) {
        System.err.println("Error al guardar los empleados: " + e.getMessage());
    }
}

public void leer() {
    try (ObjectInputStream entrada = new ObjectInputStream(new FileInputStream(FICHERO))) {
        empleados = (ArrayList<Empleado>) entrada.readObject();
        System.out.println("Se han leído " + empleados.size() + " empleados de " + FICHERO);
    } catch (FileNotFoundException e) {
        System.err.println("No existe el fichero " + FICHERO);
    } catch (IOException e) {
        System.err.println("Error al leer el fichero: " + e.getMessage());
    } catch (ClassNotFoundException e) {
        System.err.println("El fichero no contiene objetos válidos: " + e.getMessage());
    }
}
```

La clase Empleado debe implementar la interfaz Serializable

```java
public class Empleado implements Serializable{
```